



Nombre: Marco Pino

Materia: Fundamentos de la Seguridad de los Sistemas Informáticos y de la Información.

Título: Midterm - Practical

Fecha: 13/05/2026

1. Marco Teórico

- **Comparativa técnica: ¿Por qué no se deben usar algoritmos rápidos como MD5 o SHA-256 para contraseñas?**

Mucha gente cree que en informática "rápido" siempre es "mejor". Pues no siempre. Algoritmos como MD5 o SHA-256 son auténticos Ferraris; procesan información a toda pastilla.

Esto es genial para verificar un archivo pesado, pero para las contraseñas es un peligro total. Porque si un hacker nos roba la base de datos, podemos usar esa misma velocidad para probar trillones de combinaciones por segundo. Es como tener un ladrón que puede probar un millón de llaves en un minuto. Al final, va a abrir la puerta. Por eso usamos Bcrypt que es "lento" a propósito. Es como poner un bache gigante o una puerta pesada: hace que cada intento de adivinar tome su tiempo, logrando que el ataque sea tan eterno que el hacker se rinda. MD5 ya pasó a la historia. Está tan roto que hoy en día no sirve para proteger nada que de verdad quieras mantener en secreto.

Explicación del flujo de datos: ¿En qué momento se añade el Pepper y por qué no se guarda en la base de datos?

Supongamos que el Salt es la sal para la comida, ahora el Pepper es otra cosa. Este secreto se añade justo antes de procesar la contraseña y nunca se

guarda en la base de datos. Vive solo en la memoria del servidor o en un archivo de configuración (.env).

2. Desarrollo y Evidencias

Captura del código donde se configuran los modelos de SQLAlchemy.

```
class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(unique=True, index=True)
    hashed_password: str

class UserCreate(BaseModel):
    username: str
    password: str
```

Captura de las funciones de hashing donde se aprecie la integración del Pepper.

```
def get_password_hash(password: str) -> str:
    peppered = password + PEPPER
    sha_hash = hashlib.sha256(peppered.encode('utf-8')).hexdigest()

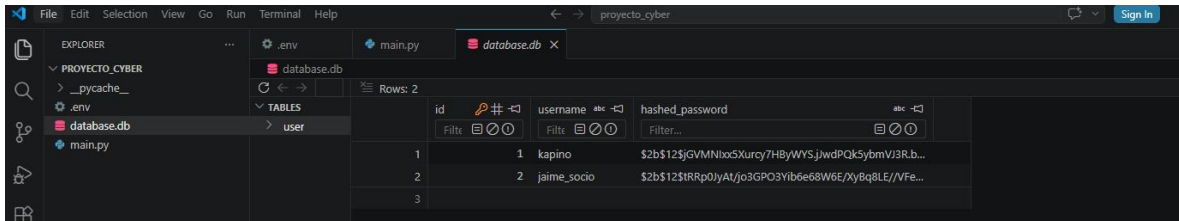
    salt = bcrypt.gensalt()
    hashed_bytes = bcrypt.hashpw(sha_hash.encode('utf-8'), salt)

    return hashed_bytes.decode('utf-8')

def verify_password(plain_p: str, hashed_p: str) -> bool:
    peppered = plain_p + PEPPER
    sha_hash = hashlib.sha256(peppered.encode('utf-8')).hexdigest()

    return bcrypt.checkpw(sha_hash.encode('utf-8'), hashed_p.encode('utf-8'))
```

Captura de pantalla de la base de datos mostrando los registros creados con sus respectivos hashes.



3. Análisis y Reflexión

¿Qué impacto tiene el Pepper en la seguridad si el atacante logra descargar el archivo .db pero no tiene acceso a las variables de entorno del servidor?

El impacto de usar un Pepper es brutal. Al ser una cadena que no está en la base de datos, el atacante va a ciegas. No sabe qué texto extra le sumamos a la contraseña antes de juntarlo. Esto rompe por completo sus bases de datos de contraseñas comunes.

¿Qué es el "costo" o "iteraciones" en el algoritmo elegido y cómo ayuda a prevenir ataques de fuerza bruta?

El costo es un parámetro que define cuántas veces se ejecuta el algoritmo internamente. En términos simples, es como decirle a la licuadora que dé más vueltas.

Prevención: Si el hardware de los hackers mejora, nosotros simplemente subimos el costo en el código para que el cálculo vuelva a ser lento para ellos, pero imperceptible para un usuario normal.

Bibliografías

- Openwall. (sf). *bcrypt password hashing for your software and your servers*. Openwall. <https://www.openwall.com/crypt/>

- Rose S. (19 de abril de 2024). *Zero Trust Architecture NIST SP 800-207*. NIST. <https://csrc.nist.gov/pubs/sp/800/207/final>
- Manisha M. (1 de mayo de 2026). *La tríada de la CIA en ciberseguridad: principios y ejemplos reales*. Atlas Systems. <https://www.atlassystems.com/blog/cia-triad-in-cybersecurity>
- Graham C. (3 de septiembre de 2024). *Salt and Pepper in Password Security*. Baeldung. <https://www.baeldung.com/cs/password-salt-pepper>
- Eren E. (22 de abril de 2026). *Building High-Performance FastAPI REST APIs for Enterprise*. Medium. <https://erenertem.medium.com/building-high-performance-fastapi-rest-apis-for-enterprise-712a107897be>
- Arunanshu B. (30 de enero de 2023). *Async Database Operations with SQLAlchemy*. DEV. <https://dev.to/arunanshub/async-database-operations-with-sqlmodel-c2o>